

Assignment 1: Basics

Ethan Chung

University of Hawaii

ECE 405: Intro to Large-Scale AI Systems

[redacted]@hawaii.edu

February 22, 2026

Contents

Byte-Pair Encoding (BPE) Tokenizer	3
Problem (unicode1): Understanding Unicode (1 point)	3
Problem (unicode2): Unicode Encodings (3 points)	3
Problem (train_bpe): BPE Tokenizer Training (15 points)	4
Problem (train_bpe_tinystories): BPE Training on TinyStories (2 points)	4
Problem (train_bpe_expts_owt): BPE Training on OpenWebText (2 points)	4
Problem (tokenizer): Implementing the tokenizer (15 points)	5
Problem (tokenizer_experiments): Experiments with tokenizers (4 points)	5
Transformer Language Model Architecture	6
Problem (linear): Implementing the linear module (1 point)	6
Problem (embedding): Implement the embedding module (1 point)	6
Problem (rmsnorm): Root Mean Square Layer Normalization (1 point)	6
Problem (positionwise_feedforward): Implement the position-wise feed-forward network (2 points)	6
Problem (rope): Implement RoPE (2 points)	6
Problem (softmax): Implement softmax (1 point)	6
Problem (scaled_dot_product_attention): Implement scaled dot-product attention (5 points)	6
Problem (multihead_self_attention): Implement causal multi-head self-attention (5 points)	6
Problem (transformer_block): Implement the Transformer block (3 points)	6
Problem (transformer_lm): Implementing the Transformer LM (3 points)	7
Problem (transformer_accounting): Transformer LM resource accounting (5 points)	7
Training a Transformer LM	8
Problem (cross_entropy): Implement Cross entropy	8
Problem (learning_rate_tuning): Tuning the learning rate (1 point)	8
Problem (adamw): Implement AdamW (2 points)	9
Problem (adamwAccounting): Resource accounting for training with AdamW (2 points)	9
Problem (learning_rate_schedule): Implement cosine learning rate schedule with warmup	11
Problem (gradient_clipping): Implement gradient clipping (1 point)	11
Training Loop	11
Problem (data_loading): Implement data loading (2 points)	11
Problem (checkpointing): Implement model checkpointing (1 point)	11

Generating Text	11
Problem (training_together): Put it together (4 points)	11
Problem (decoding): Decoding (3 points)	12
Experiments	12
Problem (experiment_log): Experiment logging (3 points)	12
Problem (learning_rate): Tune the learning rate (3 points)	12
Problem (batch_size_experiment): Batch size variations (1 point)	13
Problem (generate): Generate text (1 point)	14
Ablations	15
Problem (layer_norm_ablation): Remove RMSNorm and train (1 point)	15
Problem (pre_norm_ablation): Implement post-norm and train (1 point)	16
Problem (no_pos_emb): Implement NoPE (1 point)	17
Problem (swiglu_ablation): SwiGLU vs. SiLU (1 point)	17
Problem (main_experiment): Experiment on OWT (2 points)	18
Appendix	19
Code Availability	19
Training Logs	19

Byte-Pair Encoding (BPE) Tokenizer

Problem (unicode1): Understanding Unicode (1 point)

- (a) What Unicode character does `chr(0)` return?
`chr(0)` returns `\x00`, which is the Unicode character U+0000 or the null byte.
- (b) How does this character's string representation (`__repr__()`) differ from its printed representation?
`chr(0).__repr__()` shows the escaped value with `'\x00'`, while the printed representation is the raw character output, which is invisible and appears to show nothing.
- (c) What happens when this character occurs in text?

```
>>> chr(0)
'\x00'
>>> print(chr(0))

>>> "this is a test" + chr(0) + "string"
'this is a test\x00string'
>>> print("this is a test" + chr(0) + "string")
this is a teststring
>>> x = "a" + chr(0); print(x); x[1];
a
'\x00'
```

The string is evaluated as a null byte `\x00`, which prints nothing (although it does still exist in the string as proven by the fifth eval).

Problem (unicode2): Unicode Encodings (3 points)

- (a) What are some reasons to prefer training our tokenizer on UTF-8 encoded bytes, rather than UTF-16 or UTF-32?
UTF-8 encoded bytes range from [0,255], whereas the UTF-16 and UTF-32 would range from [0,65535] and [0,4294967295], respectively. Using either of these would negate our goal of trying to compress the sequence of integers into something manageable for token vocab to stay small.
- (b) Consider the following (incorrect) function, which is intended to decode a UTF-8 byte string into a Unicode string. Why is this function incorrect? Provide an example of an input byte string that yields incorrect results.

If I use "¥" (which is `b'\xc2\xa5'`) for the byte string, it results in the following error:

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'

>>> decode_utf8_bytes_to_str_wrong("¥".encode("utf-8"))
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xc2 in position 0:
  ↳ unexpected end of data
```

Unicode has variable-length multi-byte sequences and for symbols outside of the standard ASCII set, it will use multiple bytes to represent a single character, whereas this implementation assumes all characters = 1 byte.

- (c) Give a two byte sequence that does not decode to any Unicode character(s).

```
>>> "f".encode("utf-8")
b'\xe1\xba\x9c'

>>> b'\xe1\xba'.decode("utf-8")
UnicodeDecodeError: 'utf-8' codec can't decode bytes in position 0-1:
    ↳ unexpected end of data
```

\xe1 describes that there should be three bytes present in the string to form the character, but only two were provided in the decode, thus it errors due to a mismatch in the sequence length.

Problem (train_bpe): BPE Tokenizer Training (15 points)

Implemented in cs336_basics/train_bpe.py

Problem (train_bpe_tinystories): BPE Training on TinyStories (2 points)

- (a) Train a byte-level BPE tokenizer on the TinyStories dataset, using a maximum vocabulary size of 10,000. How many hours and memory did training take? What is the longest token in the vocabulary? Does it make sense?

Training took a total of 99.54 seconds, utilizing peak memory usage of 118.01 MB, and the longest token in the vocabulary was found to be 15 bytes long for " accomplishment". This does make sense, as it is likely the result of merging the subwords of " accomplish" and "ment" together.

- (b) What part of the tokenizer training process takes the most time?

Breaking down the total time, the longest ranked from top to bottom are:

Step	Time (s)
Pre-tokenization	55.95
Merge loop (total)	36.27
Merge updates (total)	2.90
Pair count initialization	0.31
Vocabulary initialization	0.00

The pre-tokenization step dominates, taking 55.95s out of 99.54s total. This makes sense as it involves scanning the entire dataset to split text into initial tokens before any merges can begin.

Implemented in scripts/train_bpe_dataset.py

Problem (train_bpe_expts_owt): BPE Training on OpenWebText (2 points)

- (a) Train a byte-level BPE tokenizer on the OpenWebText dataset, using a maximum vocabulary size of 32,000. What is the longest token in the vocabulary? Does it make sense?

[illegible]

- (b) Compare and contrast the tokenizer that you get training on TinyStories versus OpenWebText.

The TinyStories tokenizer learns simpler vocabulary tokens from children's stories which contain simplistic phrases, while the OpenWebText tokenizer contains a more broad and diverse vocabulary from the internet. The OWT tokenizer also captures more domain-specific subwords, like technical terms, punctuation patterns, URLs, etc.

Implemented in scripts/train_bpe_dataset.py

Problem (tokenizer): Implementing the tokenizer (15 points)

Implemented in cs336_basics/tokenizer.py

Problem (tokenizer_experiments): Experiments with tokenizers (4 points)

- (a) Sample 10 documents from TinyStories and OpenWebText. Using your previously-trained TinyStories and OpenWebText tokenizers (10K and 32K vocabulary size, respectively), encode these sampled documents into integer IDs. What is each tokenizer's compression ratio (bytes/token)?

Dataset	Total Bytes	Total Tokens	Compression Ratio
TinyStories	7,513	1,812	4.146 bytes/token
OpenWebText	38,175	8,653	4.412 bytes/token

The OpenWebText tokenizer achieves a 6.4% better compression ratio than the TinyStories tokenizer on their respective datasets, which is expected given its larger vocabulary size (32K vs. 10K) allowing it to encode more text per token.

- (b) What happens if you tokenize your OpenWebText sample with the TinyStories tokenizer? Compare the compression ratio.

Tokenizer	Total Bytes	Total Tokens	Compression Ratio
OWT tokenizer on OWT	38,175	8,653	4.412 bytes/token
TS tokenizer on OWT	36,380	11,653	3.122 bytes/token

Applying the TinyStories tokenizer to the OpenWebText sample yields a 29.2% worse compression ratio (3.122 vs. 4.412 bytes/token). This is expected, as the TinyStories tokenizer has not learned the vocabulary of more diverse web text, so it falls back to shorter and less efficient token merges.

- (c) Estimate the throughput of your tokenizer (e.g., in bytes/second). How long would it take to tokenize the Pile dataset (825GB of text)?

Tokenizer	Throughput
TinyStories	1,466,294 bytes/s (1.47 MB/s)
OpenWebText	1,242,250 bytes/s (1.24 MB/s)

Throughput was measured qualitatively by averaging the encoding over 5 runs. At 1.47 MB/s, tokenizing the 825 GB Pile dataset would take approximately 167.8 hours using the TinyStories tokenizer; at 1.24 MB/s with the OpenWebText tokenizer, approximately 198.1 hours. The OWT tokenizer is slightly slower, likely because its larger vocabulary (32K) requires searching a larger merge table on each step.

- (d) Using your TinyStories and OpenWebText tokenizers, encode the respective training and development datasets into a sequence of integer token IDs. We recommend serializing the token IDs as a NumPy array of datatype uint16. Why is uint16 an appropriate choice? uint16 stores values in the range $[0, 65535]$, which comfortably covers both vocabulary sizes (10K and 32K) and only costs 2 bytes per token. Using uint8 $[0, 255]$ would be too small to index either vocabulary, while uint32 would double the storage cost to 4 bytes per token unnecessarily.

Implemented in scripts/tokenizer_experiments.py

Transformer Language Model Architecture

Problem (linear): Implementing the linear module (1 point)

Implemented in cs336_basics/module/linear.py

Problem (embedding): Implement the embedding module (1 point)

Implemented in cs336_basics/module/embedding.py

Problem (rmsnorm): Root Mean Square Layer Normalization (1 point)

Implemented in cs336_basics/module/rmsnorm.py

Problem (positionwise_feedforward): Implement the position-wise feed-forward network (2 points)

Implemented in cs336_basics/module/swiglu.py

Problem (rope): Implement RoPE (2 points)

Implemented in cs336_basics/module/rope.py

Problem (softmax): Implement softmax (1 point)

Implemented in cs336_basics/softmax.py

Problem (scaled_dot_product_attention): Implement scaled dot-product attention (5 points)

Implemented in cs336_basics/attention.py

Problem (multihead_self_attention): Implement causal multi-head self-attention (5 points)

Implemented in cs336_basics/module/multihead_attention.py

Problem (transformer_block): Implement the Transformer block (3 points)

Implemented in cs336_basics/module/transformer_block.py

Problem (transformer_lm): Implementing the Transformer LM (3 points)

Implemented in `cs336_basics/transformer_lm.py`

Problem (transformer_accounting): Transformer LM resource accounting (5 points)

(a) Consider GPT-2 XL, which has the following configuration:

vocab_size	50,257	(V)
context_length	1,024	(T)
num_layers	48	(L)
d_model	1,600	(d)
num_heads	25	($H, d_k = d/H = 64$)
d_ff	6,400	(d_{ff})

Suppose we constructed our model using this configuration. How many trainable parameters would our model have? Assuming each parameter is represented using single-precision floating point, how much memory is required to just load this model?

Component	Parameters
Token embedding ($V \times d$)	$50,257 \times 1,600 = 80,411,200$
<i>Per transformer block ($\times L = 48$):</i>	
$q_proj, k_proj, v_proj, output_proj$ (each $d \times d$)	$4 \times d^2 = 4 \times 2,560,000 = 10,240,000$
w_1, w_3, w_2 (each $d_{\text{ff}} \times d$ or $d \times d_{\text{ff}}$)	$3 \times d d_{\text{ff}} = 3 \times 10,240,000 = 30,720,000$
RMSNorm $\times 2$ (each weight shape $(d,)$)	$2 \times d = 2 \times 1,600 = 3,200$
<i>Per block subtotal</i>	40,963,200
Final RMSNorm ($d,$)	1,600
LM head ($V \times d$)	80,411,200
Total	2,127,057,600

Deriving the formula from the per-component breakdown:

$$P = \underbrace{Vd}_{\text{embedding}} + L \underbrace{(4d^2 + 3dd_{\text{ff}} + 2d)}_{\text{per block}} + \underbrace{d}_{\text{final norm}} + \underbrace{Vd}_{\text{LM head}} = 2,127,057,600 \approx 2.13 \text{ billion parameters}$$

At 4 bytes per float32 parameter, loading the model requires $4P = 8,508,230,400$ bytes ≈ 8.51 GB.

(b) Identify the matrix multiplies required to complete a forward pass of our GPT-2 XL-shaped model. How many FLOPs do these matrix multiplies require in total? Assume that our input sequence has context_length tokens.

Per transformer block ($\times L$ blocks):

Operation	Description	Dimensions	FLOPs per block
Q projection	Input $\rightarrow$ queries via W_Q	$(T, d) \times (d, d)$	$2Td^2 = 5.24 \times 10^9$
K projection	Input $\rightarrow$ keys via W_K	$(T, d) \times (d, d)$	$2Td^2 = 5.24 \times 10^9$
V projection	Input $\rightarrow$ values via W_V	$(T, d) \times (d, d)$	$2Td^2 = 5.24 \times 10^9$
$QK^\top$ (scores)	$\sum_{h=1}^H (T, d_k) \times (d_k, T)$	$H \times (T, d_k) \times (d_k, T)$	$2T^2d = 3.36 \times 10^9$
Attn $\times V$ (output)	$\sum_{h=1}^H (T, T) \times (T, d_k)$	$H \times (T, T) \times (T, d_k)$	$2T^2d = 3.36 \times 10^9$
Output projection	Concat heads $\rightarrow W_O$	$(T, d) \times (d, d)$	$2Td^2 = 5.24 \times 10^9$
SwiGLU W_1 (gate branch)	Input $\rightarrow d_{\text{ff}}$ via W_1	$(T, d) \times (d, d_{\text{ff}})$	$2Td d_{\text{ff}} = 20.97 \times 10^9$
SwiGLU W_3 (linear branch)	Input $\rightarrow d_{\text{ff}}$ via W_3	$(T, d) \times (d, d_{\text{ff}})$	$2Td d_{\text{ff}} = 20.97 \times 10^9$
SwiGLU W_2 (output)	$d_{\text{ff}} \rightarrow$ output via W_2	$(T, d_{\text{ff}}) \times (d_{\text{ff}}, d)$	$2Td_{\text{ff}}d = 20.97 \times 10^9$

Additionally, adding the single LM head which takes hidden state $\rightarrow$ vocab logits, of shape $(T, d) \times (d, V)$, the FLOPs per block is $2TdV = 1.65 \times 10^{11}$, and thus the total is:

$$\text{Total} = L(8Td^2 + 4T^2d + 6Tdd_{\text{ff}}) + 2TdV \approx 4.513 \times 10^{12} \text{ FLOPs}$$

- (c) Based on your analysis above, which parts of the model require the most FLOPs?

The SwiGLU feed-forward network consumes 66.9% of total FLOPs (3,019.8B of 4,513B), since it requires three $(T, d) \times (d, d_{\text{ff}})$ matrix multiplies per layer.

- (d) Repeat your analysis with GPT-2 small (12 layers, 768 d_model, 12 heads), GPT-2 medium (24 layers, 1024 d_model, 16 heads), and GPT-2 large (36 layers, 1280 d_model, 20 heads). As the model size increases, which parts of the Transformer LM take up proportionally more or less of the total FLOPs?

Component	Small	Medium	Large	XL
Total FLOPs	0.35 TF	1.03 TF	2.26 TF	4.51 TF
Attn projections (Q/K/V/O)	16.5%	20.0%	21.5%	22.3%
Attn core ($QK^\top + AV$)	11.1%	10.0%	8.6%	7.1%
FFN (all layers)	49.8%	59.9%	64.2%	66.9%
LM head	22.6%	10.2%	5.8%	3.7%

As model size increases, the FFN dominates more ($\sim 50\% \rightarrow \sim 67\%$), since FFN cost scales as $O(Ld^2T)$ while the LM head scales only as $O(TdV)$, making the fixed-vocabulary head relatively cheaper as d and L grow. The attention core products $O(LT^2d)$ shrink proportionally because T is fixed while d and L grow, whereas the $O(Ld^2T)$ attention projections grow in step with the FFN.

- (e) Take GPT-2 XL and increase the context length to 16,384. How does the total FLOPs for one forward pass change? How do the relative contribution of FLOPs of the model components change?

Increasing context length from 1,024 to 16,384 increases total FLOPs by approximately $33\times$ (from 4.51 TFLOPs to $\approx 1.50 \times 10^{14}$ FLOPs ≈ 150 TFLOPs), because the attention $QK^\top$ and AV products scale quadratically with T . As a result, the attention core ($QK^\top + AV$) grows from 7.1% to 55.2% of total FLOPs, FFN drops from 66.9% to 32.3%, and attention projections drop from 22.3% to 10.8%.

Training a Transformer LM

Problem (cross_entropy): Implement Cross entropy

Implemented in `cs336_basics/cross_entropy.py`

Problem (learning_rate_tuning): Tuning the learning rate (1 point)

Run the SGD example with three other values for the learning rate: 1e1, 1e2, and 1e3, for just 10 training iterations. What happens with the loss for each of these learning rates? Does it decay faster, slower, or does it diverge (i.e., increase over the course of training)?

With `lr=1e1` the loss decays faster than the baseline (`lr=1`), reaching 3.25 after 10 iterations vs. 20.0; with `lr=1e2` the loss converges extremely rapidly, collapsing to near-zero by iteration 4 (1.49×10^{-16}). With `lr=1e3` the loss diverges from 24.2 to 2.24×10^{18} over 10 iterations.

Implemented in `scripts/sgd_learning_rates.py`

Problem (adamw): Implement AdamW (2 points)

Implemented in `cs336_basics/adamw.py`

Problem (adamwAccounting): Resource accounting for training with AdamW (2 points)

- (a) How much peak memory does running AdamW require? Decompose your answer based on the memory usage of the parameters, activations, gradients, and optimizer state. Express your answer in terms of the `batch_size` and the model hyperparameters (`vocab_size`, `context_length`, `num_layers`, `d_model`, `num_heads`). Assume $d_{\text{ff}} = 4 \times d_{\text{model}}$.

Activations.

Operation	Output shape	Elements
<i>Per transformer block ($\times L$):</i>		
RMSNorm ln1	(B, T, d)	BTd
Q projection	(B, T, d)	BTd
K projection	(B, T, d)	BTd
V projection	(B, T, d)	BTd
QK^T scores	(B, H, T, T)	BHT^2
Softmax	(B, H, T, T)	BHT^2
Weighted sum ($\text{Attn} \times V$)	(B, T, d)	BTd
Output projection	(B, T, d)	BTd
RMSNorm ln2	(B, T, d)	BTd
W_1 output	(B, T, d_{ff})	$4BTd$
SiLU output	(B, T, d_{ff})	$4BTd$
W_2 output	(B, T, d)	BTd
<i>Per block subtotal</i>		$17BTd + 2BHT^2$
<i>Outside blocks:</i>		
Final RMSNorm	(B, T, d)	BTd
LM head	(B, T, V)	BTV
Cross-entropy softmax	(BT, V)	BTV

$$A = L(17BTd + 2BHT^2) + BT(d + 2V)$$

Parameters.

Component	Parameter count
Token embedding ($V \times d$)	Vd
<i>Per transformer block ($\times L$):</i>	
Q, K, V, O projections (each $d \times d$)	$4d^2$
SiLU FFN: W_1, W_2 (each $d \times d_{\text{ff}}$)	$2d d_{\text{ff}} = 8d^2$
RMSNorm $\times 2$	$2d$
Final RMSNorm	d
LM head ($V \times d$)	Vd

$$P = 2Vd + L(12d^2 + 2d) + d$$

Memory. Each float32 value is 4 bytes.

Memory component	Bytes
Parameters	$4P$
Gradients	$4P$
Optimizer state (m and v)	$8P$
Activations	$4A$
Total	$16P + 4A$

Substituting P and A :

$$\text{Total} = 16[2Vd + L(12d^2 + 2d) + d] + 4B[L(17Td + 2HT^2) + T(d + 2V)] \text{ bytes}$$

- (b) Instantiate your answer for a GPT-2 XL-shaped model ($V = 50,257$, $T = 1,024$, $L = 48$, $d = 1,600$, $H = 25$) to get an expression that only depends on `batch_size`. What is the maximum batch size you can use and still fit within 80 GB memory?

Parameter count:

$$P = 2(50257)(1600) + 48(12(1600)^2 + 2(1600)) + 1600 = 1,635,537,600$$

Fixed memory (parameters + gradients + optimizer state):

$$16P = 16 \times 1,635,537,600 = 26,168,601,600 \text{ bytes} \approx 26.17 \text{ GB}$$

Per-sequence activation coefficient:

$$\begin{aligned} L(17Td + 2HT^2) + T(d + 2V) &= 48(17(1024)(1600) + 2(25)(1024)^2) + 1024(1600 + 2(50257)) \\ &= 3,958,081,536 \end{aligned}$$

$$4 \times 3,958,081,536 = 15,832,326,144 \text{ bytes per sequence} \approx 15.83 \text{ GB per sequence}$$

Memory as a function of batch size B :

$$\text{Memory} = 15,832,326,144 \cdot B + 26,168,601,600 \text{ bytes} \approx 15.83B + 26.17 \text{ GB}$$

Maximum batch size for 80 GB:

$$B_{\max} = \left\lfloor \frac{80 \times 10^9 - 26,168,601,600}{15,832,326,144} \right\rfloor = \lfloor 3.40 \rfloor = 3$$

$$B = 3 \text{ requires } \approx 73.7 \text{ GB}; B = 4 \text{ would require } \approx 89.5 \text{ GB}.$$

- (c) How many FLOPs does running one step of AdamW take?

AdamW performs only elementwise operations. Per parameter:

Line	FLOPs/param
$m \leftarrow \beta_1 m + (1 - \beta_1)g$	3
$v \leftarrow \beta_2 v + (1 - \beta_2)g^2$	4
$\theta \leftarrow \theta - \alpha_t \cdot m / (\sqrt{v} + \varepsilon)$	5
$\theta \leftarrow \theta - \alpha \lambda \theta$ (weight decay)	2
Total	14

$$\text{AdamW FLOPs per step} = 14P = 14 \times 1,635,537,600 = 2.289 \times 10^{11}$$

- (d) Model FLOPs utilization (MFU) is defined as the ratio of observed throughput (tokens per second) relative to the hardware’s theoretical peak FLOP throughput. An NVIDIA A100 GPU has a theoretical peak of 19.5 teraFLOP/s for float32. Assuming 50% MFU, how long would it take to train GPT-2 XL for 400K steps at batch size 1024 on a single A100? Assume the backward pass has twice the FLOPs of the forward pass.

Forward FLOPs per sequence (SiLU FFN, $d_{\text{ff}} = 4d$):

$$\text{Per block: } 8Td^2 + 4T^2d + 4Td d_{\text{ff}} = 8Td^2 + 4T^2d + 16Td^2 = 24Td^2 + 4T^2d$$

$$\text{Forward FLOPs} = L(24Td^2 + 4T^2d) + 2TdV \approx 3.507 \times 10^{12} \text{ FLOPs/sequence}$$

FLOPs per training step (forward + backward = $3 \times$ forward):

Component	FLOPs
Per sequence (fwd + bwd)	$3 \times 3.507 \times 10^{12} = 1.052 \times 10^{13}$
Per step ($B = 1024$)	$1024 \times 1.052 \times 10^{13} = 1.077 \times 10^{16}$
AdamW overhead	2.29×10^{10} (negligible)

Total training FLOPs and time:

$$\text{Total FLOPs} = 400,000 \times 1.077 \times 10^{16} \approx 4.31 \times 10^{21}$$

$$\text{Time} = \frac{4.31 \times 10^{21}}{9.75 \times 10^{12}} \approx 4.42 \times 10^8 \text{ s} \approx \mathbf{5,115 \text{ days} \approx \mathbf{14 \text{ years}}}$$

Problem (learning_rate_schedule): Implement cosine learning rate schedule with warmup

Implemented in cs336_basics/lr_schedule.py

Problem (gradient_clipping): Implement gradient clipping (1 point)

Implemented in cs336_basics/gradient_clipping.py

Training Loop

Problem (data_loading): Implement data loading (2 points)

Implemented in cs336_basics/data.py

Problem (checkpointing): Implement model checkpointing (1 point)

Implemented in cs336_basics/serialization.py

Generating Text

Problem (training_together): Put it together (4 points)

Although the assignment specifies memory-efficient loading via `np.memmap`, I found that using `np.memmap` caused the GPU to stall waiting on CPU I/O (GPU utilization oscillating between 0% and 100%), which would have extended training to ~40 hours. Since the encoded TinyStories and OWT datasets fit comfortably in system memory, I load them directly with `np.load`. I

also added `torch.set_float32_matmul_precision('high')` and `torch.compile`, which increased throughput from ~ 1 – 2 iterations/s to ~ 10 iterations/s and was able to complete a training run in 1 hour (before hyper-parameter tuning).

Implemented in `cs336_basics/train.py`

Problem (decoding): Decoding (3 points)

Implemented in `cs336_basics/generation.py`

Experiments

Problem (experiment_log): Experiment logging (3 points)

All experiment logs are available on the GitHub repository under the `logs/` folder. Runs are also tracked on Weights & Biases at <https://wandb.ai/echung32-ece405/projects>, which contains projects for the OpenWebText run, TinyStories ablation studies, TinyStories batch sweep, and TinyStories learning rate sweep.

Problem (learning_rate): Tune the learning rate (3 points)

- (a) The learning rates were swept between $1e-5$ and $1e-2$ over 40K steps (batch size 128, 2,000-step cosine warmup), showing steady improvement with increasing LR and diminishing returns past $1e-3$, with $1e-2$ achieving the best validation loss of 1.2493 (≤ 1.45 target) and no divergence. Although $1e-2$ should have been used for subsequent runs, a logging misread led to using $3e-3$ instead, which is within approximately 0.86% of the best validation loss (1.2600 vs. 1.2493) and should not meaningfully affect performance.

Max LR	Final Val Loss	Final Val PPL	Wall-clock Time
$1e-5$	2.1901	8.94	88m 6s
$3e-5$	1.7935	6.01	86m 51s
$1e-4$	1.5060	4.51	86m 56s
$3e-4$	1.3558	3.88	87m 3s
$6e-4$	1.3058	3.69	86m 55s
$1e-3$	1.2837	3.61	87m 3s
$3e-3$	1.2600	3.53	86m 55s
$1e-2$	1.2493	3.49	86m 51s

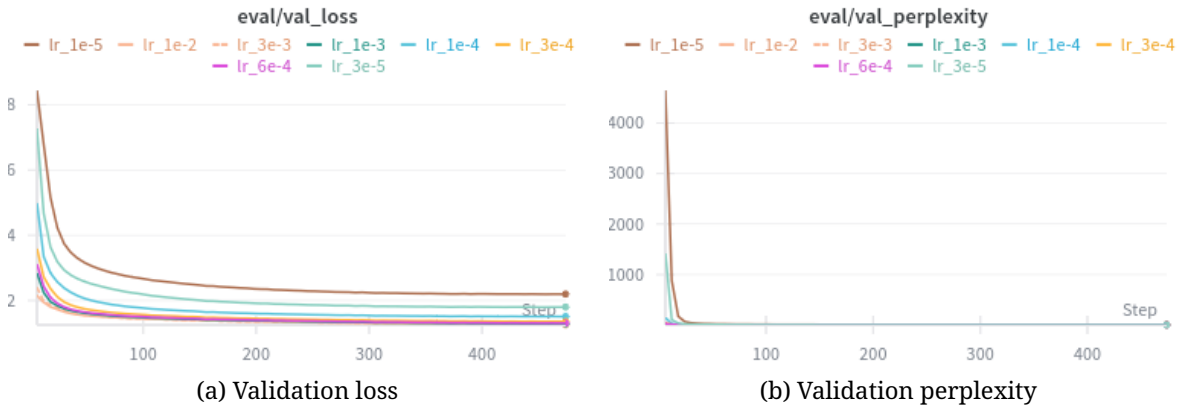


Figure 1: Validation loss and perplexity across learning rate sweep (40K iterations, batch size 128).

- (b) Increasing the learning rate progressively showed faster early loss reduction as we approached $1e-2$, indicating proximity to the stability boundary while still maintaining convergence. Pushing the rate far beyond this (e.g., $1e2$) produced highly erratic, non-convergent validation curves characteristic of divergence. Thus, the best-performing rate ($1e-2$) lies just below the edge of stability.

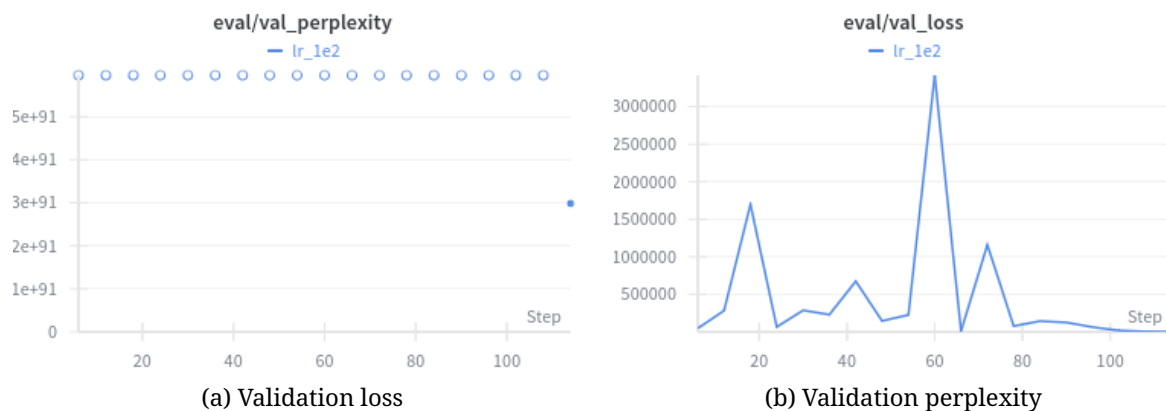


Figure 2: Validation loss and perplexity across learning rate sweep (40K iterations, batch size 128).

Implemented in `slurm/train_learning_rate_sweep.slurm`

Problem (batch_size_experiment): Batch size variations (1 point)

All batch-size runs used max LR $3e-3$, cosine decay with 2,000-iteration warmup, and were trained for 10,000 iterations on TinyStories.

Batch size	Final Val Loss	Final Val PPL	Wall-clock Time
1	3.3106	27.40	3m 6s
8	2.4593	11.70	3m 11s
32	1.5403	4.67	7m 18s
64	1.4293	4.18	12m 23s
128	1.3563	3.88	22m 24s
256	1.3044	3.69	43m 38s

Batch sizes from 1 to 256 were evaluated for 10K steps (max LR $3e-3$, 2,000-step cosine warmup), showing steadily improved validation loss as batch size increased due to reduced gradient variance, with performance improving from 3.31 (batch 1) to 1.30 (batch 256). Although 128 and 256 achieved slightly better losses, batch size 64 was selected for future runs because it offered a strong trade-off between performance and efficiency (1.43 val loss in 12 minutes) compared to the marginal improvement at batch 256 (1.30 val loss in 43 minutes).

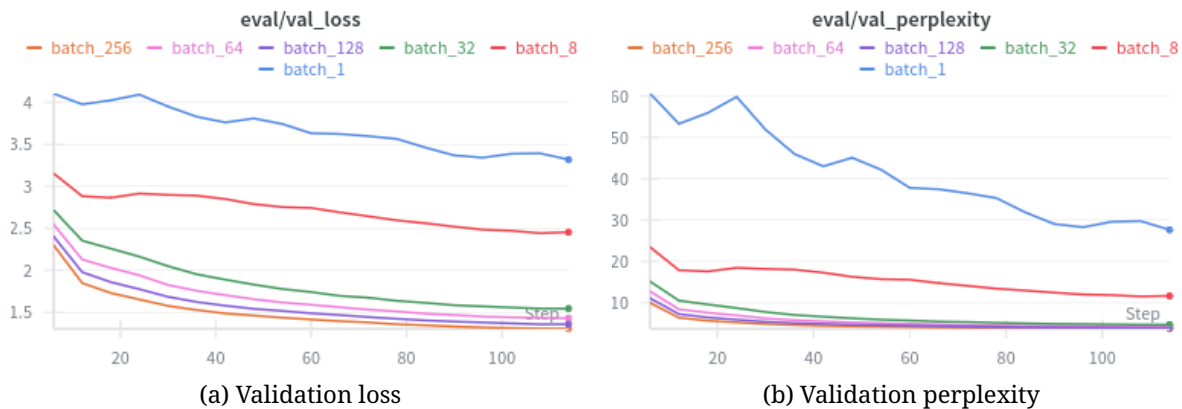


Figure 3: Validation loss and perplexity across batch sizes (10K iterations, max LR $3e-3$).

Implemented in `slurm/train_batch_sweep.slurm`

Problem (generate): Generate text (1 point)

All the text generated below uses the checkpoint files for batch size 64, learning rate $3e-3$.

Once upon a time, there was a little boy named Tim. Tim was very happy and loved to play with his toys. One day, Tim found a big box of clothes in his room. He put them on and went to play with his friends. Tim saw his friend, Sam, and asked, "What are you doing?" Sam replied, "I am playing with this clothes, but I have to separate them. I want to play with my toys." Tim thought for a moment and said, "I recommend we play a game with my toys." Tim and Sam played a game of tag. They ran around the room, chasing each other and laughing. They had so much fun playing together. The sun started to set, and it was time for Sam to go home. Tim and Sam said goodbye and promised to play together again soon. Tim went to bed, excited to play with his friends again tomorrow. And they all knew they would have more fun days together.<|endoftext|>

`dataset=TinyStories, temp=0.8, top-p=0.9, prompt="Once upon a time,"`

The equation 3×12 evaluates to the world. The sleepy little girl was filled with joy and excitement. She was filled with joy and joy. She knew that the sleepy little girl had worked so hard and she wanted to explore more. The little girl packed her things and headed off. On the way, she came across a bright blue river. It was beautiful and the little girl loved it. She jumped in and swam around the river, exploring and enjoying the beautiful beauty. The end.<|endoftext|>

`dataset=TinyStories, temp=0.8, top-p=0.9, prompt="The equation  $3 \times 12$  evaluates to"`

We can see that the fluency of the text is highly dependent on what the dataset was trained on; using anything other than storytelling terms on will likely not result in a coherent response, but using "Once upon a time," is more likely to produce a coherent response that makes some sense. In the case of "The equation 3×12 evaluates to", the model has no understanding of arithmetic and immediately drifts back to storytelling with no relation to the start.

As an exercise, the prompt was also evaluated at double the temperature (temp 1.6), which resulted in the following output:

Once upon a time, an Jenny wanted to help it get mole apples. They filledions everySara read flap of expectingy logs until there would step Oneundayleum parade everyday. thoughts answered up intrigued all the moral of the compared names of itsurt whole novel from one car landscapes and Alice found an enormous green rotten deliver outside car 0 gun broke on rainy d clips sandwiches instead of apples crayon Fightingmyence today." From ine Please Mama, Mama ideas Bobo often Why wouldery Mia who would pack Bearnved buying all was healthy pupils's favorite color sandcastle fl cob Joke enjoy as love yourself. Daisy have tested the3 guests play in gor0w chefs,erg Cats in enthusiastic hall trouble twice, twelvedged shortsetings, pretending animalsPete spaceshiping quarrel posesMrs wellrough Sunny being kind. Anna kept sear temperature promises Grandma showed her how patience wool always faaud ears exclaimed to taps blossouushed fair, and mayri Turtle Near nights shares funny piano inviting Sam. Sally understood what thick peace sounded like when all by beadsPenny with medals by her smooth dress focus. From this experience, Anna gazed out for anReadyDon lighting the positiveimLuna opera attitudeivermered tall. Withwards everybody helping Tylerbers permissionhe seen hugging drippingAnn inBetty's scent, making

dataset=TinyStories, temp=1.6, top-p=0.9, prompt="Once upon a time,"

It can be seen that doubling the temperature dramatically degrades output quality: the model produces incoherent word salad with broken grammar and invented tokens. High temperature flattens the probability distribution over the vocabulary, making low-probability tokens nearly as likely to be sampled as high-probability ones, which compounds across each generation step into nonsensical text.

Implemented in cs336_basics/generation.py

Ablations

All ablation variants were trained for 10,000 iterations with batch size 64, cosine decay schedule, and 2,000-iteration warmup, which matches the batch sweep baseline.

Problem (layer_norm_ablation): Remove RMSNorm and train (1 point)

Variant	Max LR	Final Val Loss	Final Val PPL	Time
Pre-norm (baseline)	3e-3	1.4293	4.18	22m 24s
No norm (diverged)	3e-3	NaN	—	11m 38s
No norm (retry)	1e-5	3.0166	20.42	13m 23s

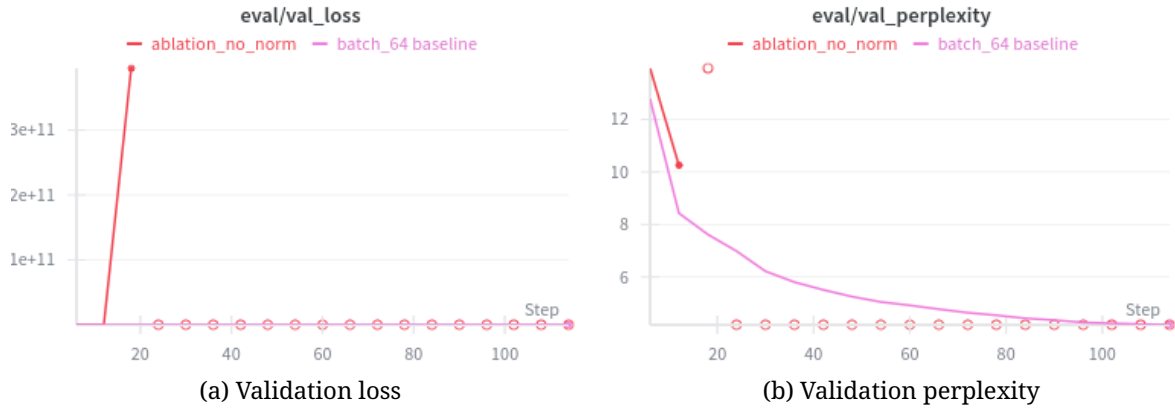


Figure 4: Validation loss and perplexity for the no-norm ablation at 3e-3 learning rate. The pre-norm baseline is not visible in the loss plot as the no-norm run diverges and explodes.

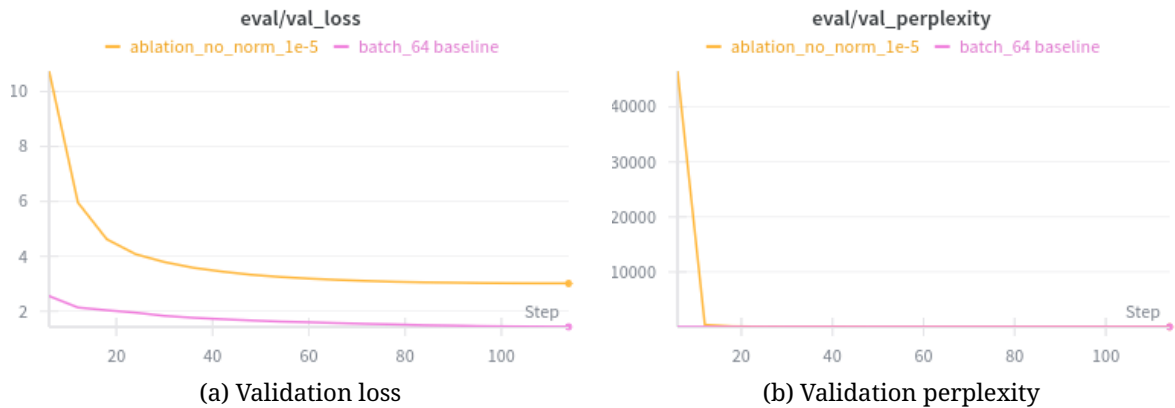


Figure 5: Validation loss and perplexity for the no-norm ablation at 1e-5 learning rate, which does not explode but converges much more slowly than the pre-norm baseline.

Without RMSNorm, training at the optimal learning rate (3e-3) diverges rapidly. The loss spiked to 2.85×10^8 at iteration 1500 and became NaN by iteration 1600, with all subsequent evaluations producing NaN. Retrying with a drastically reduced learning rate (1e-5) prevented the explosion, but the model converged to a much worse final validation loss of 3.0166 (ppl 20.42), compared to 1.4293 (ppl 4.18) for the pre-norm baseline.

Implemented in `cs336_basics/ablation`

Problem (pre_norm_ablation): Implement post-norm and train (1 point)

Variant	Final Val Loss	Final Val PPL	Time
Pre-norm (baseline)	1.4293	4.18	22m 24s
Post-norm	1.4716	4.36	12m 2s

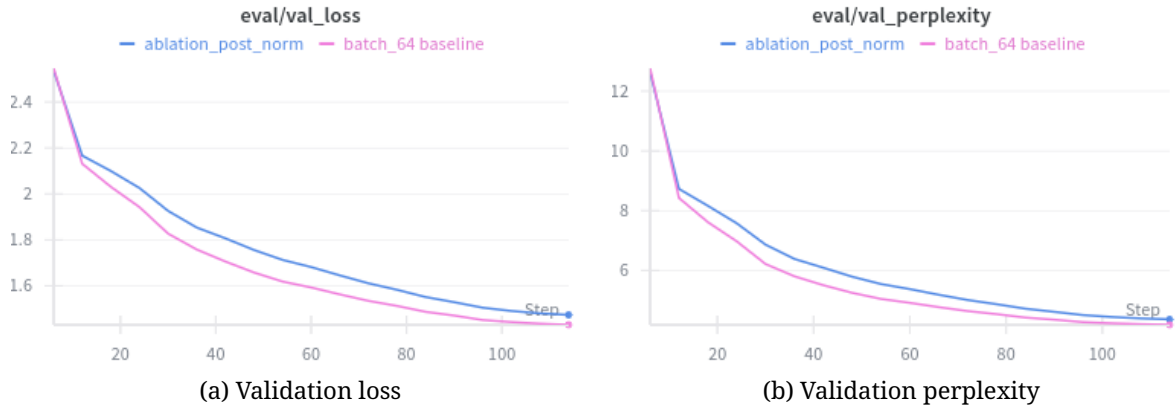


Figure 6: Validation loss and perplexity comparing pre-norm and post-norm variants (10K iterations, max LR $3e-3$).

Implemented in `cs336_basics/ablation`

Problem (no_pos_emb): Implement NoPE (1 point)

Variant	Final Val Loss	Final Val PPL	Time
RoPE (baseline)	1.4293	4.18	22m 24s
NoPE	1.4981	4.47	11m 52s

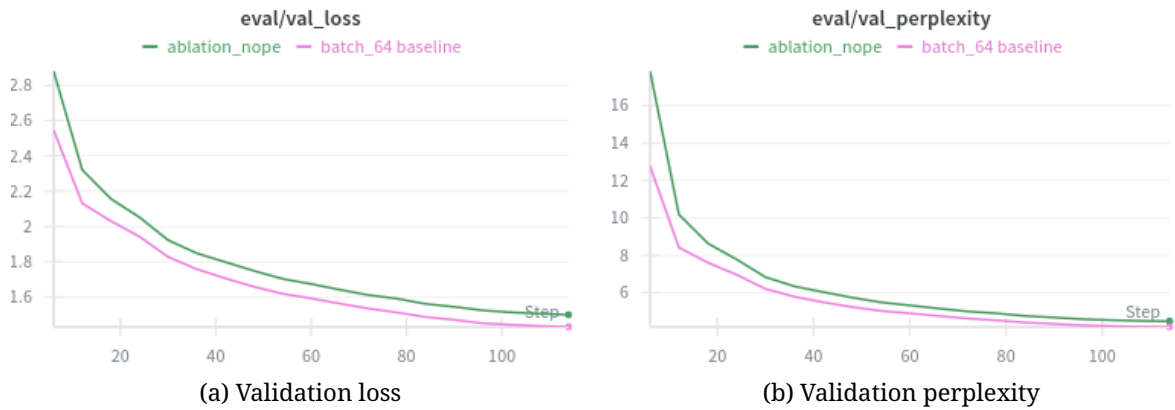


Figure 7: Validation loss and perplexity comparing RoPE and NoPE variants (10K iterations, max LR $3e-3$).

Implemented in `cs336_basics/ablation`

Problem (swiglu_ablation): SwiGLU vs. SiLU (1 point)

Variant	Final Val Loss	Final Val PPL	Time
SwiGLU (baseline)	1.4293	4.18	22m 24s
SiLU	1.4466	4.25	11m 52s

The SiLU model achieves a final val loss of 1.4466 (ppl 4.25), only 1.2% worse than the SwiGLU baseline. Both converged smoothly at $3e-3$. The minimal difference suggests that at this model scale (22.7M SwiGLU vs. 22.8M SiLU parameters, 10K iterations, TinyStories), the gating mechanism of SwiGLU provides only a marginal benefit over a simpler SiLU activation.

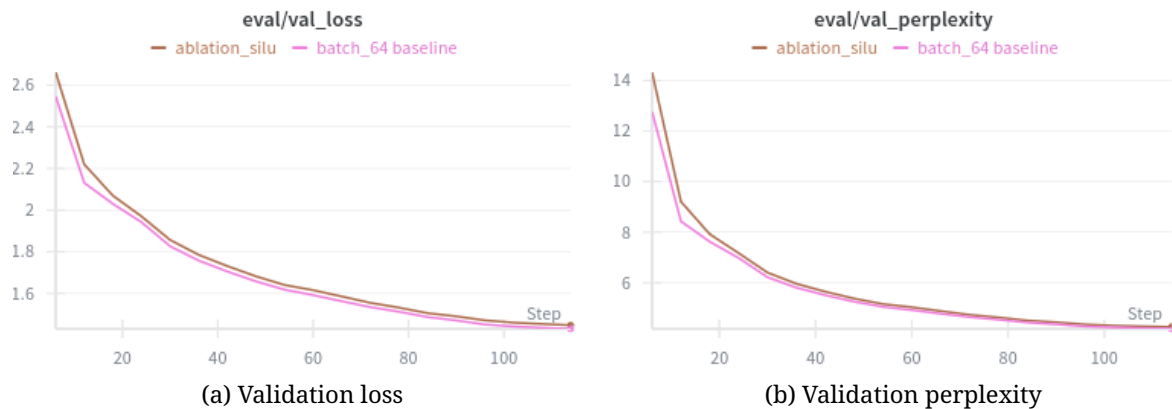


Figure 8: Validation loss and perplexity comparing SwiGLU and SiLU variants (10K iterations, max LR $3e-3$).

Implemented in `cs336_basics/ablation`

Problem (main_experiment): Experiment on OWT (2 points)

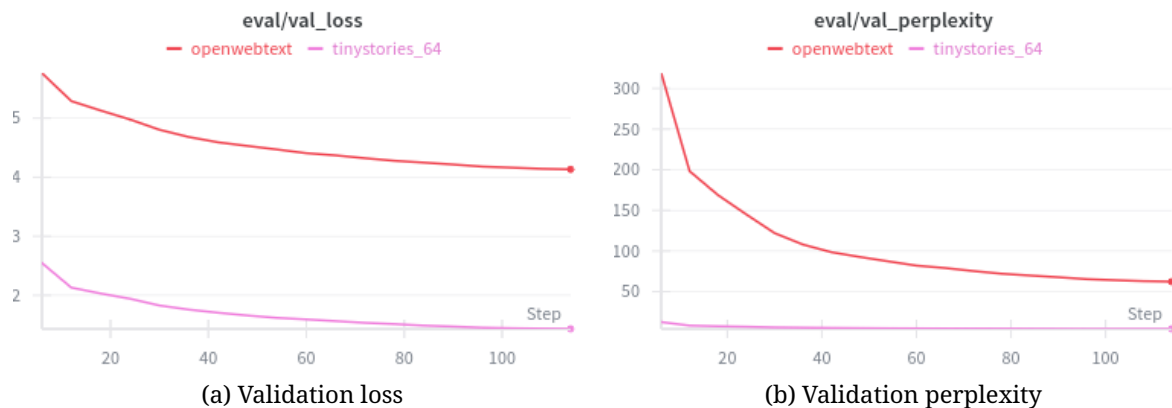


Figure 9: Validation loss and perplexity for OWT vs. TinyStories baseline (10K iterations, max LR $3e-3$, batch size 64).

After 10K iterations, the OpenWebText (OWT) model has much higher validation loss and perplexity than TinyStories, and the gap remains roughly constant throughout training, indicating OWT is far from converged under the same computational budget. The generated OWT samples are stylistically web-like but semantically incoherent, with repetitive and hollow phrasing (e.g., “the function of the function”). Despite identical architecture and training steps, the model must spread its limited capacity across many domains and has not trained long enough to capture OWT’s broader distribution.

Once upon a time, we have to say that we have the upper hand that we are going to have to wait for.
 I believe that we will have to ask ourselves to give a more accurate description, even if they're not saying that we are, in fact, just the right people to express that opinion, but we have to give the impression that we have a way to know that we will be out of touch.
 "We will also like to look at what we will say to you," I said.

I didn't think we would have a great time to do the same. But they had to do a lot of things, for the majority of the time, not to do it. I'm not worried that we will get more intelligence.
I think this is a good time to do this. But for a while, I think that's enough to say the least.
But the fact is, it seems like every day.
So, I feel like the fact that we know the United States are in good shape. I think they have a really good understanding of what we're doing.
We know the United States is too often in the realm of certainty, and I think that's just as much more appropriate. We know

dataset=OpenWebText, temp=0.8, top-p=0.9, prompt="Once upon a time,"

The equation 3×12 evaluates to any extent the difference between "approach" and "due-value".
Note that there is a sense of fact that the function of the function is the same as the function to be defined by the function and the function. For example, if the function is a function to have the function, which should be "likely" and "to have the function that you can find".
The function of the function of the function is called when the function takes a function. In that case, it is called function which functions it. For example, the function is called "over" or "in" - which is the function of the function (1) and the function (2) is called "in".
The function can be used to define a function and returns the function of the function.
When the function takes a function, it doesn't matter.
The function should function as a function for the function.
The function takes a function that calls it's function. It needs the function to do that, which is why it does.
It is crucial for the function to execute this function.
It is important to note that it is important to keep this function in place.
It is

dataset=OpenWebText, temp=0.8, top-p=0.9, prompt="The equation 3×12 evaluates to"
Implemented in cs336_basics/generation.py

Appendix

Code Availability

All code mentioned in this report is available at github.com/echung32/ece405-assignment1-basics

Training Logs

All training logs and ablation studies are available at Weights & Biases:

- TinyStories learning rate sweep: wandb.ai/echung32-ece405/ece405-tinystories-lrsweep
- TinyStories batch sweep: wandb.ai/echung32-ece405/ece405-tinystories-batchsweep
- TinyStories ablation studies: wandb.ai/echung32-ece405/ece405-tinystories-ablation
- OpenWebText: wandb.ai/echung32-ece405/ece405-openwebtext